NEOENERGIA PERNAMBUCO · PROJETO 4: DADOS

Verificação Automatizada de Fotos de Leitura

Arquitetura da solução técnica: fluxo de processamento, componentes, modelo de dados, camada de IA e tecnologias adotadas.

Documento técnico da solução

Setembro de 2026 · versão para apresentação

PONTO DE PARTIDA

O que a solução faz e como foi desenhada

Um pipeline que recebe os lotes diários de leitura (CSV + fotos), filtra automaticamente o que é seguro decidir e entrega aos auditores apenas a zona de incerteza. Cada passo é registrado como evento, o que torna o fluxo mensurável de ponta a ponta.

1. Simplicidade antes de sofisticação

Heurística e modelo pré-treinado antes de treino customizado. Monólito antes de microserviços. Fila leve antes de orquestrador pesado.

2. Consistência antes de OCR pleno

O sistema não "lê" o medidor do zero. Verifica se a foto é coerente com a leitura e a ocorrência já declaradas.

3. Faseamento com gate de métrica

Cada fase só avança se a anterior atingiu o critério de sucesso definido. O calendário não decide, a métrica decide.

4. Humano no que é incerto

Auto-decisão só onde a confiança é alta e medida. A decisão humana vira rótulo e retroalimenta o modelo.

2

filtros automáticos antes do auditor: qualidade da imagem e consistência foto × dado declarado

4

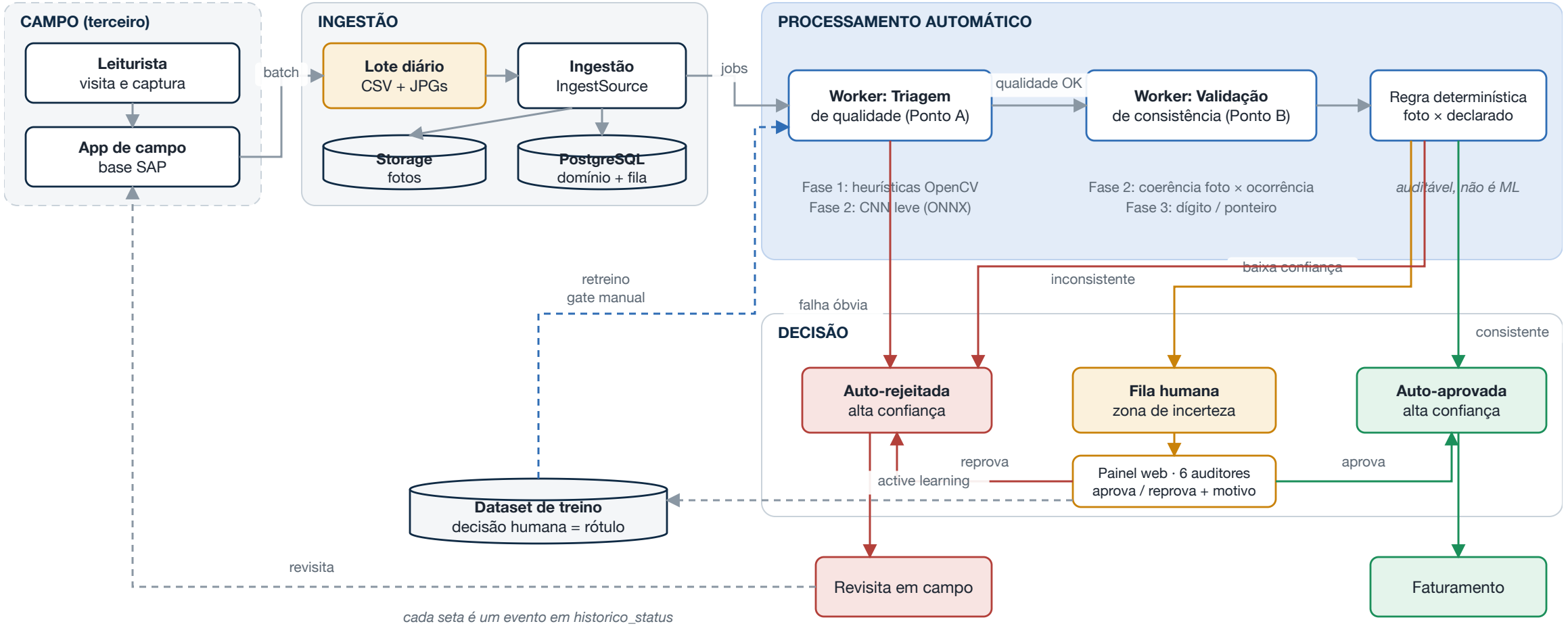
interfaces plugáveis que isolam decisões de infraestrutura: origem de lote, storage, autenticação e detector

100%

das transições de estado registradas em histórico append-only, base da camada analítica

VISÃO MACRO

Fluxo ponta a ponta: do lote em campo ao veredito



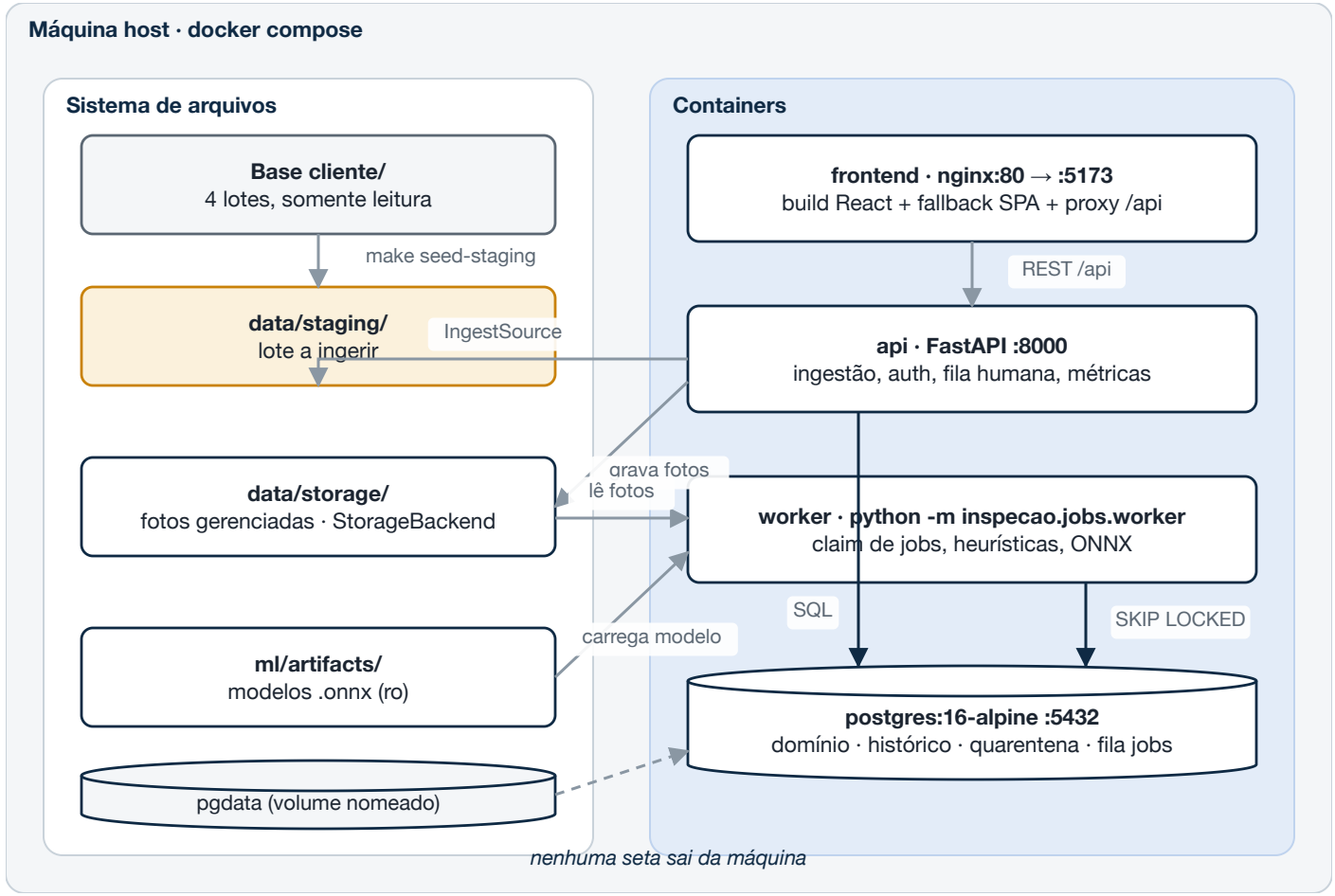
TECNOLOGIAS

Stack adotada e o motivo de cada escolha

Camada	Tecnologia	Por quê
Linguagem e ambiente	Python 3.12 fixado via `uv`	Stack de visão (torch, opencv, onnxruntime) ainda sem wheels estáveis para 3.14. Lockfile reproduzível.
API	FastAPI + Pydantic + Uvicorn	API fina que só orquestra. Tipagem forte gera o OpenAPI de onde saem os tipos do frontend.
Banco	PostgreSQL 16	Único banco para domínio, histórico, quarentena e fila. Volume por lote é trivial.
ORM e migrações	SQLAlchemy 2 + Alembic	Esquema versionado e reversível. Inserção em lote para bater a meta de ingestão.
Fila de jobs	Tabela `jobs` no Postgres com `SELECT … FOR UPDATE SKIP LOCKED`	Fila transacional junto ao domínio, sem broker adicional. Redis + RQ é o caminho de upgrade.
Storage de fotos	Adapter `StorageBackend`: filesystem local hoje, S3 / MinIO em produção	Nenhum chamador conhece o backend. Gravação atômica e proteção contra path traversal.
Inferência	OpenCV (heurísticas) + ONNX Runtime em workers dedicados	Desacopla API e modelo. Modelo carregado uma vez por worker.
Treino	PyTorch + torchvision, scikit-learn, exportação ONNX	Treino fora do caminho de produção. Paridade torch × onnxruntime verificada antes de publicar.
Frontend	React 18 + TypeScript + Vite + Mantine + TanStack Query + React Router	SPA com autenticação robusta e rotas protegidas por papel. Cache e revalidação da fila sem estado global.
Serviço web	nginx	Serve o build estático, faz fallback de SPA e proxy de `/api` sob o mesmo domínio.
Orquestração	Docker Compose	Quatro serviços em um arquivo. Kubernetes só se volume ou equipe crescerem.

IMPLANTAÇÃO

Topologia de execução: quatro serviços, um compose



Sem dependência externa em runtime

Nenhuma seta sai da máquina. Rede externa só na instalação de dependências. O mesmo compose sobe em notebook de desenvolvimento e em servidor on-premise.

Volumes

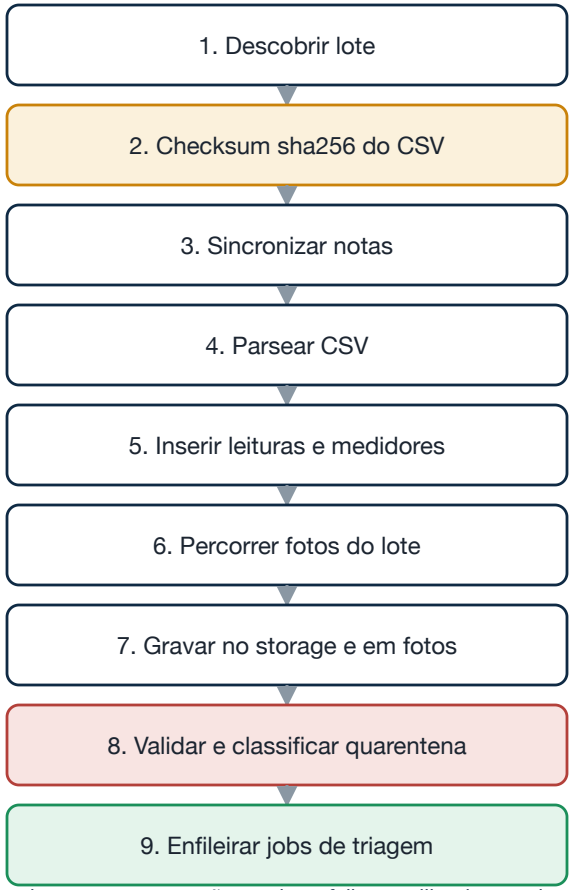
- `data/staging/` : lote a ingerir, somente leitura pela API
- `data/storage/` : fotos gerenciadas pelo `StorageBackend`
- `ml/artifacts/` : modelos `.onnx` montados como somente leitura nos workers
- `pgdata` : volume nomeado do Postgres

Operação

- Healthcheck do Postgres condiciona a subida da API e do worker
- `/api/health` checa banco, storage e presença do modelo
- Política `restart: unless-stopped` em todos os serviços
- Worker sobe pelo profile `completo` a partir do marco M3

PIPELINE DE DADOS

Ingestão: do lote bruto ao domínio consistente



tudo em uma transação por lote: falha = rollback completo

Contrato do CSV

- Delimitador `;`, cabeçalho fixo de 4 colunas validado antes de qualquer inserção
- Sentinela `NA` tratada como ausência de valor, não como texto
- Coluna de medidor separada por `/`: até 8 medidores por linha, identificadores alfanuméricos preservados como texto
- Nome da foto casado por regex: extrai `id_externo` e `sequencia`

Garantias

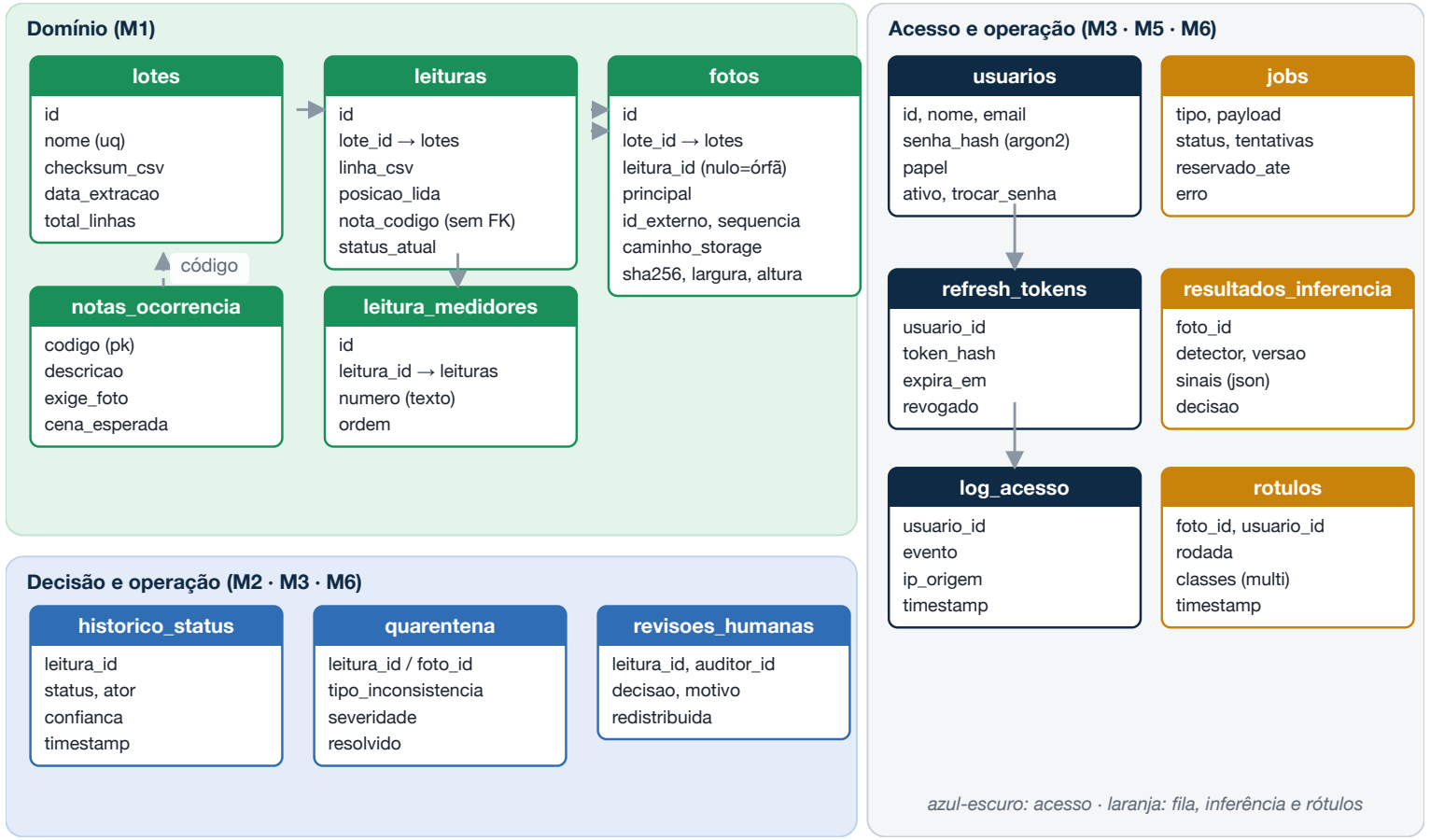
- **Idempotência** por checksum sha256 do CSV. Reingerir o mesmo lote não altera nada; `--reprocessar` substitui de forma explícita
- **Performance**: inserção em lote de 2.000 linhas e dimensões lidas direto do cabeçalho JPEG, sem decodificar a imagem
- **Quarentena, não bloqueio**: inconsistências viram registro classificado e o lote segue

Ponto de extensão

A origem vive atrás de `IngestSource`. Hoje é pasta em disco (`BatchFolderSource`). Um webhook do sistema de campo entra como nova implementação sem tocar o resto do pipeline.

MODELO DE DADOS

Esquema relacional: domínio, decisão, operação e acesso



Decisões de modelagem

- Relação leitura → foto vive em `fotos.leitura_id` + `principal`, evitando chave estrangeira circular. Foto órfã tem `leitura_id` nulo
- `leituras.nota_codigo` sem chave estrangeira: código não mapeado entra e vai para quarentena, nunca é descartado
- Medidor sempre texto: existem identificadores como `F50081`

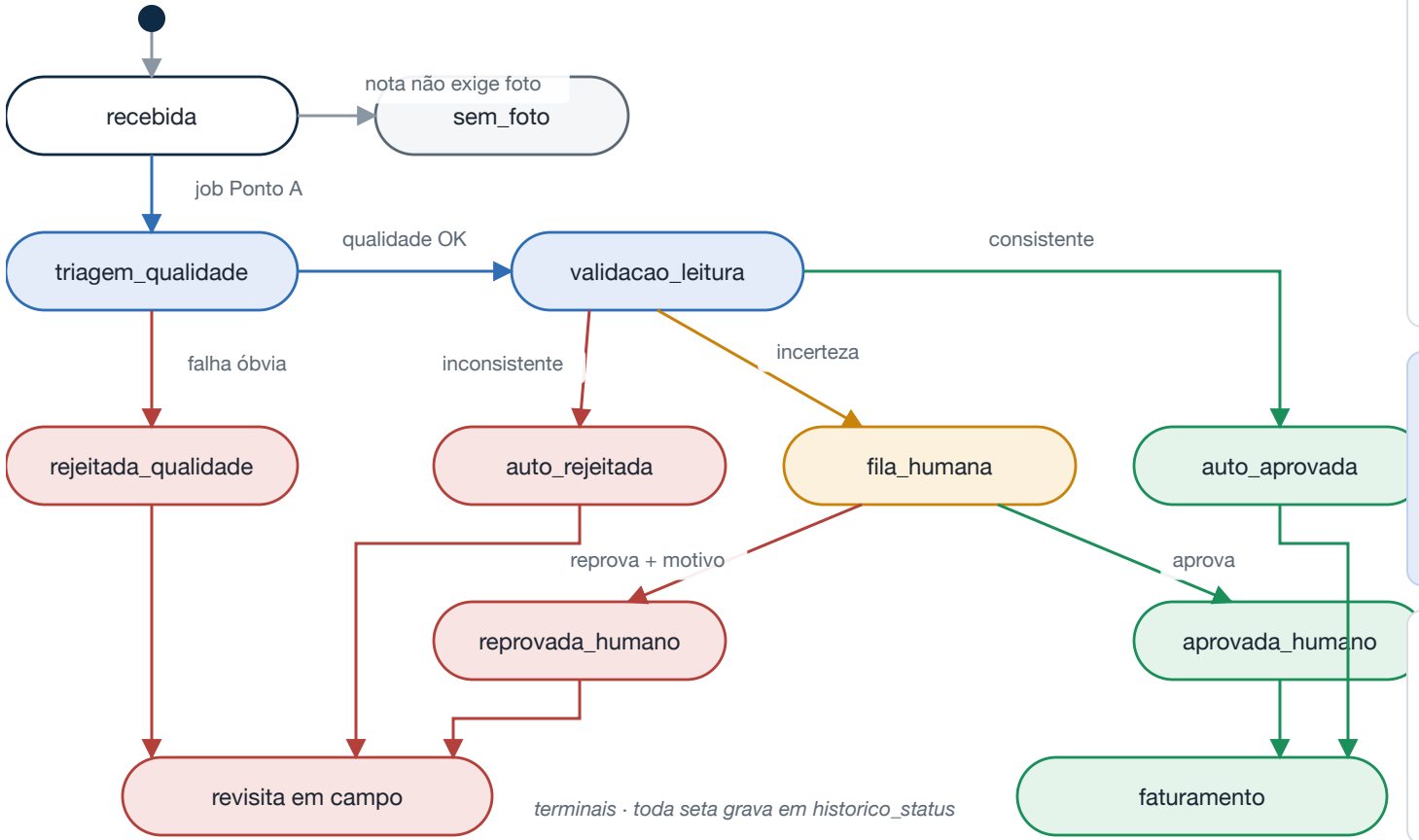
Append-only por padrão

- `historico_status` : toda transição, com ator e confiança
- `log_acesso` : todo login, falho ou não
- `revisoes_humanas` e `rotulos` : decisões nunca sobrescritas

É isso que permite medir taxa de foto inutilizável, latência até o veredito e produtividade da auditoria, números que hoje não existem.

CICLO DE VIDA DA LEITURA

Máquina de estados: uma função de transição, um histórico



Regras

- `status_atual` só é escrito pela função `transicionar()`, que valida a transição contra a tabela permitida. Um teste varre o código para garantir isso
- Cada transição grava linha em `historico_status` com ator (worker, modelo ou auditor), confiança e timestamp
- Nunca há `UPDATE` destrutivo de status sem histórico

Estados terminais

Faturamento recebe auto-aprovadas e aprovadas por humano. **Revisita** recebe rejeitadas por qualidade, auto-rejeitadas e reprovadas por humano.

Estado especial

`sem_foto` cobre leituras cuja nota não exige evidência fotográfica. Saem do fluxo de inferência sem entrar na fila humana.

CAMADA DE INTELIGÊNCIA

Dois pontos de decisão automática, um loop de aprendizado

Ponto A filtra qualidade da imagem. Ponto B verifica consistência entre a foto e o que foi declarado.
A decisão humana sobre o que sobra alimenta o retreino, sempre com gate manual.

PONTO A

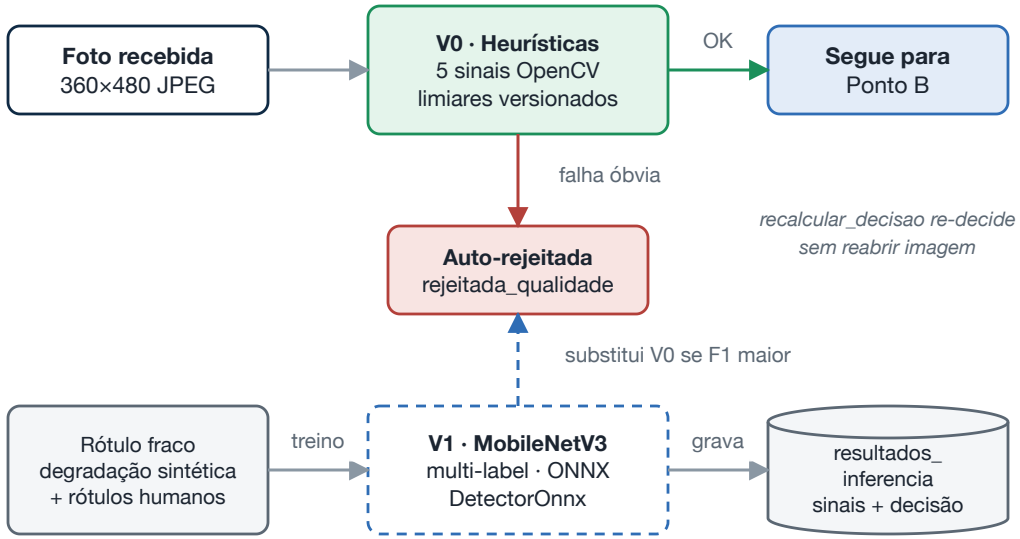
Triagem de qualidade: heurística primeiro, modelo depois

V0 · Heurísticas de visão clássica (sem treino)

- **Nitidez**: variância do Laplaciano abaixo de limiar indica fora de foco
- **Exposição e reflexo**: picos de saturação no histograma indicam glare
- **Contraste e uniformidade**: display apagado ou tampa embaçada
- **Presença de display**: região retangular de alto contraste no quadro
- Limiares calibrados nas 12.340 imagens reais e fixados em política versionada

V1 · Classificador multi-label (CNN leve)

- MobileNetV3 com transfer learning, entrada na resolução nativa 360×480
- Classes: glare, fora de foco, tampa suja ou embaçada, enquadramento incorreto, display apagado
- **Estágio 1**: pré-treino com rótulo fraco por degradação sintética (blur, overlay de glare, downscale)
- **Estágio 2**: fine-tune nos rótulos humanos consolidados
- Exportado para ONNX, servido por `DetectorOnnx` nos workers

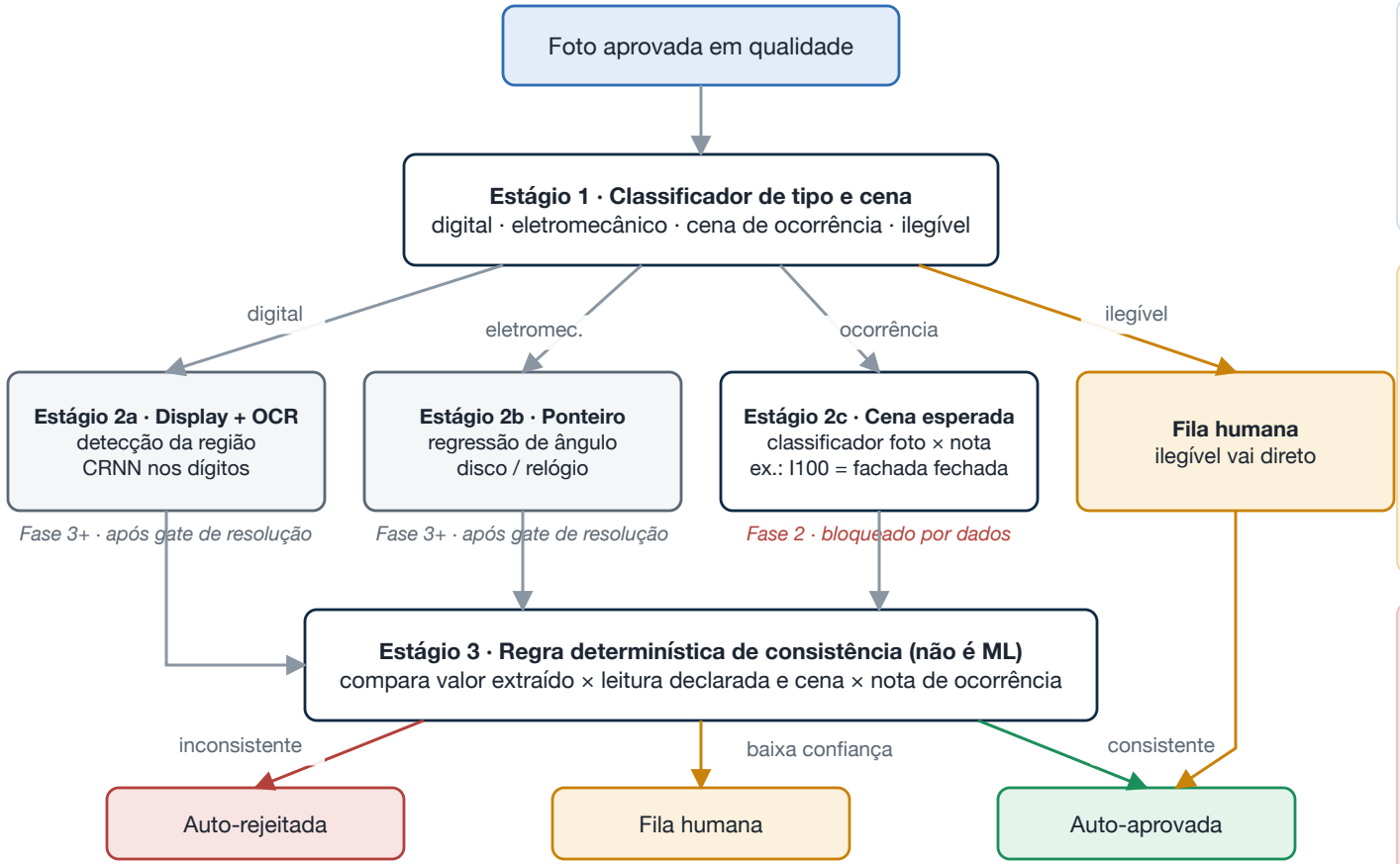


Gate de aceite

Entre 5% e 15% do volume auto-rejeitado com **zero** falso positivo confirmado nas 200 imagens de extremo inspecionadas. O modelo V1 só substitui a heurística se bater o macro-F1 dela no mesmo conjunto rotulado. Se não bater, não entra.

PONTO B

Validação de consistência: três estágios, roteamento condicional



Por que não OCR pleno

O sistema não precisa ler o medidor do zero. Compara o que extrai da foto com a leitura e a ocorrência já declaradas. O Estágio 3 é uma regra determinística, não um modelo, o que a torna auditável.

Gate de viabilidade de resolução

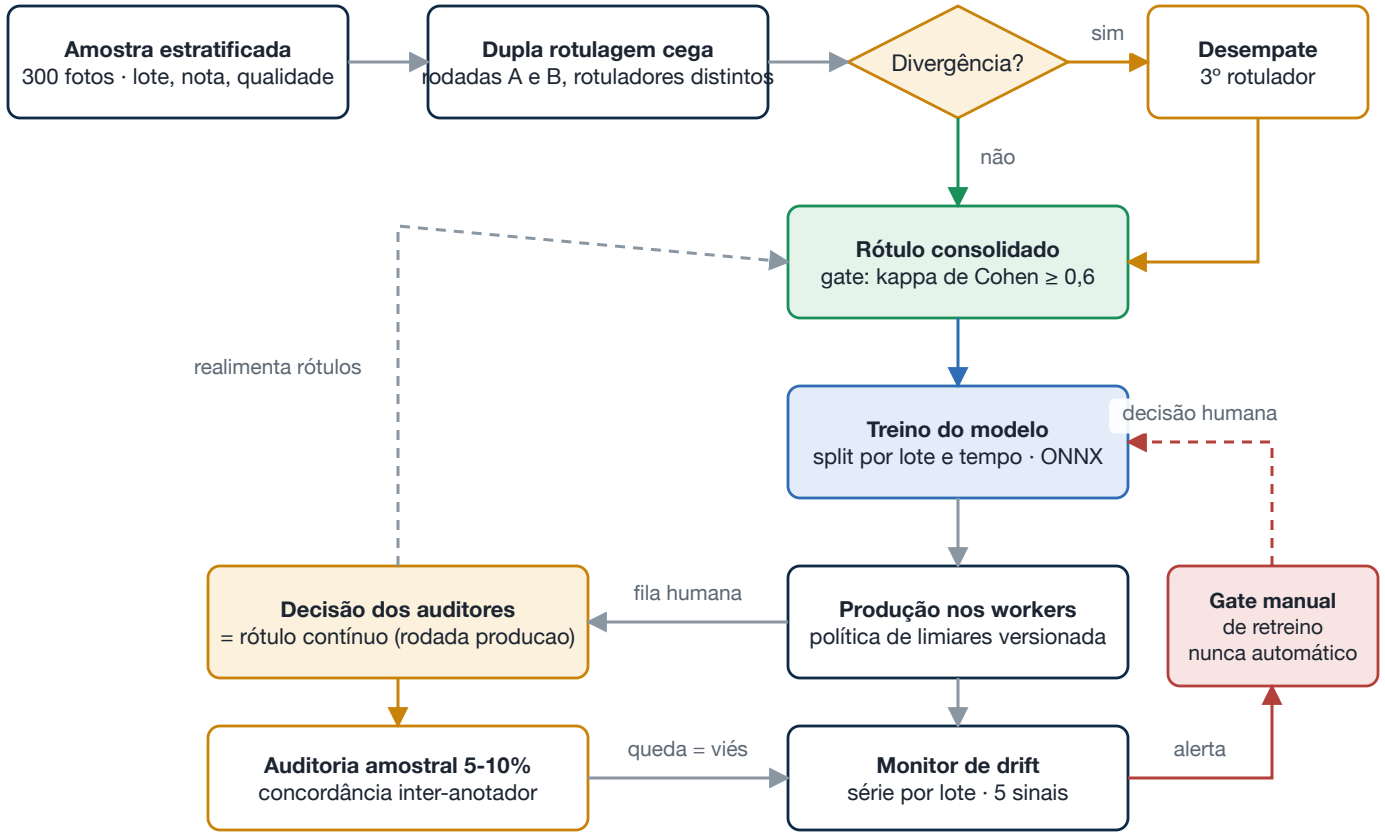
Antes de investir em OCR de dígito ou regressão de ponteiro, um spike de um dia roda OCR pronto (Tesseract, EasyOCR) sobre 150 fotos legíveis com gabarito manual. Se a taxa for insuficiente em 360×480, a saída é pedir maior resolução ao cliente, não sofisticar o modelo.

Restrição de dados atual

Dos 61 códigos de ocorrência, 19 aparecem na base e 96% das notas esperam a mesma cena: o display. O Estágio 2c depende de amostra dirigida com cenários físicos distintos, já registrada como pendência de entrada.

MLOPS

Ground truth, active learning e monitoramento de drift



Bootstrap sem viés

- 300 fotos em amostra estratificada por lote, nota e faixa de qualidade
- Dupla rotulagem cega com cegueira garantida no servidor
- Gate: kappa de Cohen igual ou acima de 0,6. Abaixo disso, as definições de classe são revisadas e a rodada refeita

Salvaguardas de treino

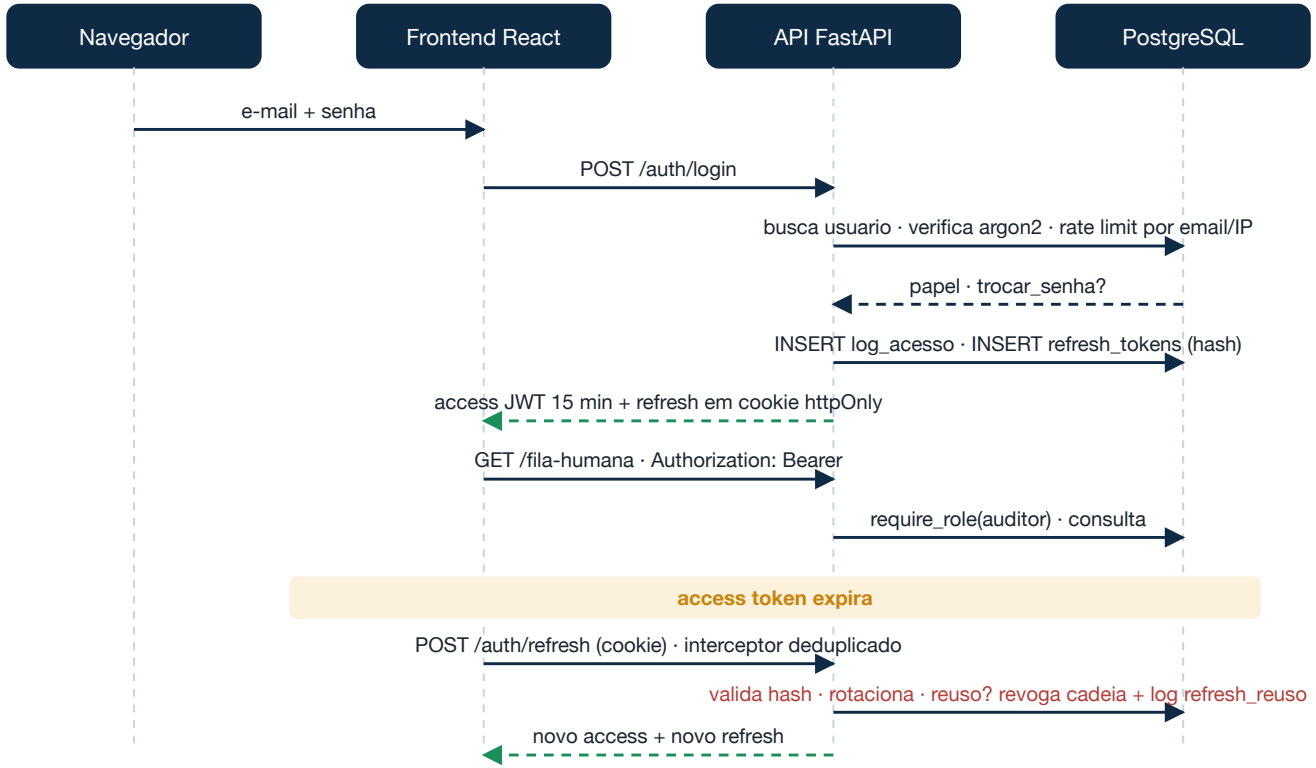
- Split treino/validação por lote e tempo, nunca aleatório
- Métricas por classe com intervalo de confiança, nunca só o agregado
- Treino reproduzível por semente

Drift e retreino

- Série por lote: distribuição de defeitos, taxa de rejeição, confiança média, medidor não visto
- Auditoria amostral contínua de 5% a 10% mede concordância inter-anotador
- Retreino **nunca** automático. Alerta dispara decisão humana

SEGURANÇA E ACESSO

Autenticação, papéis e trilha de auditoria



Mecanismos

- Access token JWT de 15 minutos. Refresh token opaco de 7 dias em cookie `httpOnly` e `secure`, com hash no banco
- Rotação de refresh com detecção de reuso: reapresentar token já usado revoga toda a cadeia do usuário
- Senhas com argon2. Rate limiting de login por e-mail e IP. Troca de senha forçada no primeiro acesso

RBAC hierárquico

Papel	Acessa
`auditor`	Fila humana e decisão
`supervisor`	+ métricas, drift, quarentena
`admin`	+ gestão de usuários e papéis

Cada rota valida o papel no servidor, via dependency do FastAPI.

Evolução para SSO

Login atrás de `AuthProvider`. Um provedor OpenID Connect (Entra ID ou outro IdP) entra sem reescrever backend ou frontend.

INTERFACES

API REST e painel de auditoria

Rota	Função · papel mínimo
`POST /auth/login · /refresh · /logout`	Emissão, renovação e revogação de tokens
`GET /auth/me`	Usuário e papel, para o frontend ajustar a interface
`POST /ingest/batch`	Dispara processamento de lote em staging · admin
`POST /ingest/webhook`	Pronto e desativado até integração com o sistema de campo
`GET /leituras/{id}`	Detalhe da leitura com histórico completo · auditor
`GET /fila-humana`	Lista paginada e filtrável, com reserva de item por 10 min · auditor
`POST /fila-humana/{id}/decisao`	Aprova ou reprova com motivo em taxonomia fechada · auditor
`GET /rotulagem/proxima · POST /rotulagem/{foto_id}`	Rotulagem dupla-cega, cegueira garantida no servidor · auditor
`GET /quarentena`	Inconsistências de dados pendentes · supervisor
`GET /metricas/qualidade`	Funil por status, auto-decisão, defeitos, latências, produtividade · supervisor
`GET /metricas/drift`	Série por lote dos sinais de drift · supervisor

Tipos TypeScript gerados a partir do OpenAPI: mudança de contrato quebra o build antes de quebrar em produção.

Telas do painel

- **Login** e troca de senha obrigatória
- **Fila de revisão**: visualizador com zoom e pan, painel com leitura e nota declaradas, decisão com atalhos de teclado e pré-carregamento da próxima imagem
- **Rotulagem**: modo cego com as definições operacionais das 5 classes visíveis
- **Quarentena e métricas**: funil, baseline e drift por lote
- **Usuários**: papéis e ativação

Mecânica do cliente

- Interceptor de refresh com deduplicação de chamadas concorrentes
- `RotaProtegida` por papel mínimo
- TanStack Query revalida fila e métricas sem estado global manual
- nginx serve o build, faz fallback de SPA e proxy de `/api`

EVOLUÇÃO

Pontos de extensão: o que muda sem reescrever o sistema

`IngestSource`

Hoje: pasta em disco com CSV e fotos.

Depois: webhook ou API do sistema de campo, quando negociado com o terceiro.

`StorageBackend`

Hoje: filesystem local com gravação atômica.

Depois: S3 ou MinIO, conforme contrato de infraestrutura do cliente.

`AuthProvider`

Hoje: senha local com argon2 e JWT próprio.

Depois: OpenID Connect com o provedor de identidade corporativo.

`Detector`

Hoje: `DetectorNulo` roteia tudo para humano; heurísticas OpenCV na sequência.

Depois: `DetectorOnnx` com o modelo treinado, trocado por configuração.

Fila de jobs

Tabela no Postgres com claim atômico, backoff, recuperação de job órfão e desligamento limpo em SIGTERM. Se o volume exigir, Redis + RQ entra como implementação alternativa da mesma interface de enfileiramento.

Política de decisão versionada

Limiares de auto-rejeição e auto-aprovação vivem em arquivo versionado. O job `recalcular_decisao` re-decide o histórico sem reabrir imagem, o que permite ajustar política e medir o efeito sobre lotes já processados.

Auto-aprovação desligada por padrão

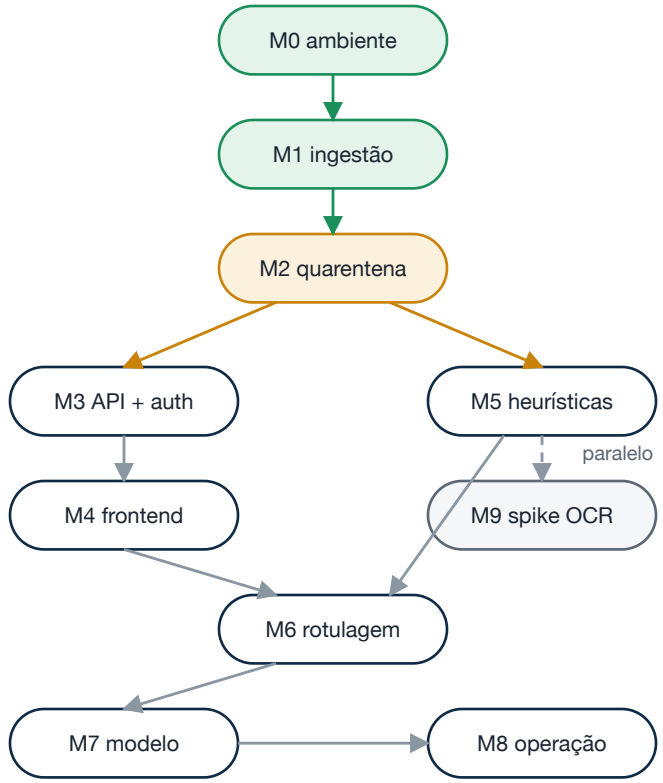
A interface mostra o motivo estatístico: quantidade de rótulos acumulados contra os 600 necessários para estimar a taxa de falso-negativo com o intervalo de confiança exigido. Liga-se por decisão de supervisor, não por deploy.

ROADMAP TÉCNICO

Marcos de execução e estado atual

Marco	Entrega	Critério de aceite verificável	Estado
M0	Ambiente e esqueleto	Compose sobe 4 serviços, `/health` 200, dados do cliente fora do git	CONCLUÍDO
M1	Ingestão e domínio	13.669 leituras, 12.340 fotos, 61 notas em < 90 s. Reingestão idempotente	CONCLUÍDO
M2	Validação e quarentena	Contagens exatas por tipo de inconsistência em teste de regressão	PRÓXIMO
M3	API, autenticação e fila	2 workers × 500 jobs, cada job exatamente uma vez. RBAC e rotação testados	PLANEJADO
M4	Frontend de revisão	20 decisões por teclado gravadas em revisões e histórico	PLANEJADO
M5	Heurísticas de qualidade	5% a 15% auto-rejeitado, zero falso positivo em 200 imagens	PLANEJADO
M6	Rotulagem e ground truth	300 fotos consolidadas, kappa ≥ 0,6	PLANEJADO
M7	Modelo de qualidade	Macro-F1 acima da heurística, paridade ONNX, treino reproduzível	PLANEJADO
M8	Operação, limiares e drift	Baseline real nos 4 lotes, auditoria amostral ativa	PLANEJADO
M9	Spike de OCR (paralelo)	Veredito de viabilidade fundamentado em 360×480	PLANEJADO
—	Classificador de cena	Depende de amostra dirigida do cliente	BLOQUEADO

Dependências entre marcos



M5 anda em paralelo com M3 e M4 · M9 não bloqueia nada

SÍNTESE

Uma arquitetura que decide o que é seguro e mede tudo o resto

Monólito Python com Postgres como única dependência de estado. Filtros de qualidade e consistência antes do auditor. Interfaces plugáveis para origem, storage, autenticação e detector. Cada evento registrado, cada fase com gate de métrica, retreino sempre com decisão humana.